

گزارش جامع عملکرد: توسعه و یکپارچه سازی سیستم های حسگری وای فای از پردازش پایه RSSI تا شبکه های مش CSI و پردازش لبه ISAC

امیرعلی یزدیانی
دانشجوی مهندسی برق، دانشگاه صنعتی شریف
شماره دانشجویی: ۴۰۲۱۰۲۷۶۵

۳ شهریور ۱۴۰۵

چکیده اجرایی گزارش عملکرد

این سند، گزارش جامع فعالیت های انجام شده در راستای توسعه سیستم های سنجش فضایی مبتنی بر امواج وای فای است. مسیر توسعه در سه فاز متوالی طی شده است: آغاز کار با پردازشگر پایه RSSI در محیط پایتون، گذار به استخراج دقیق CSI با زبان Rust در بستر شبکه های مش چندنوده (RuView)، و در نهایت رسیدن به معماری مستقل لبه (Edge ISAC) با استفاده از پلتفرم ESpectre. در این مستند، تمامی جزئیات پیاده سازی، کدهای راه اندازی و چالش های فنی هر سه فاز به صورت پیوسته مدون گردیده است.

فهرست مطالب

۳	۱	مقدمه و چشم انداز تکاملی پروژه
۴	۲	فاز اول: سنجش پایه مبتنی بر RSSI در محیط لینوکس (RuView V1)
۳	۱۰۲	آماده سازی محیط Python
۴	۲۰۲	استخراج و پردازش RSSI
۴	۳۰۲	پایش لحظه ای (Live Monitoring)
۴	۴۰۲	تحلیل استراتژیک: چرا از فاز یک عبور کردیم؟
۵	۳	فاز دوم: شبکه مش CSI با زبان Rust (RuView V2)
۵	۱۰۳	راه اندازی محیط Rust و سرور پردازشی
۵	۲۰۳	آماده سازی ESP-IDF و پیکربندی گره های ESP32
۵	۱۰۲۰۳	روش سنتی کامپایل (menuconfig)
۶	۲۰۲۰۳	روش نوین و سریع (Provision.py)
۶	۳۰۳	چالش های استقرار شبکه و راهکارها
۶	۴۰۳	تحلیل استراتژیک: چرا نیازمند فاز سوم شدیم؟
۷	۴	فاز سوم: پردازش لبه متمرکز (Edge ISAC) با ESpectre
۷	۱۰۴	کدنویسی بدون C++ (Zero-Bloat Architecture)
۷	۱۰۱۰۴	پیکربندی سخت افزاری برد
۸	۲۰۴	الگوریتم های پردازشی در لبه
۸	۳۰۴	یکپارچه سازی با Home Assistant در بستر Docker

۱ مقدمه و چشم انداز تکاملی پروژه

هدف اصلی این مجموعه اقدامات، تبدیل سیگنال‌های عادی وای‌فای به یک سیستم سنجش فضایی بدون نیاز به دوربین یا ابزارهای پوشیدنی است تا بتوان حضور، حرکت و علائم حیاتی را پایش کرد. برای رسیدن به یک سیستم پایدار صنعتی، فرآیند تحقیق و توسعه در سه مرحله برنامه‌ریزی و اجرا شد که هر مرحله، پاسخی به محدودیت‌های مرحله پیشین خود بود.

۲ فاز اول: سنجش پایه مبتنی بر RSSI در محیط لینوکس (RuView V1)

در ابتدای کار، هدف یادگیری روند «جمع‌آوری داده»، «استخراج ویژگی» و «طبقه‌بندی وضعیت حضور» بود. در این فاز نیازی به سخت‌افزار جانبی نبود و تنها از کارت شبکه لپ‌تاپ استفاده گردید.

۱.۲ آماده‌سازی محیط Python

ابتدا مخزن پروژه کلون شده و محیط مجازی پایتون راه‌اندازی گردید:

```
git.RuView/ruvnet/com.github://https clone git
v1/archive/RuView cd
venv. venv m- python3
activate/bin/venv. source
txt.lock-requirements r- install pip/bin/venv.
uvicorn fastapi psutil install pip/bin/venv.
```

به دلیل اجرای پروژه در لینوکس، در کدهای سورس ماژول WindowsWifiCollector به LinuxWifiCollector تغییر یافت و رابط شبکه سیستم جایگزین شد:

```
(0.2=sample_rate_hz , "wlx002e2d2091fa="interface(LinuxWifiCollector = collector
```

۲.۲ استخراج و پردازش RSSI

ابتدا با استفاده از دستور iwconfig پارامترهای رابط شبکه (wlx002e2d2091fa) بررسی شد:

```
iwconfig v1$/archive/ruview/WifiSensing/Amirali/Desktop:~/epo@hpds
"WiFi-Sharif:"ESSID 11.802 IEEE wlx002e2d2091fa
:35D:8E2:D604:18: :Point Access GHz 412.2:Frequency Managed:Mode
dBm -56=level Signal =54/70Quality Link
```

سپس اسکریپت test36.py برای ثبت RSSI و تست حرکت تدوین گردید:

```
time ,sys import
LinuxWifiCollector import rssi_collector.sensing.src from
RssiFeatureExtractor import feature_extractor.sensing.src from
```

```

PresenceClassifier import classifier.sensing.src from

(0.2=sample_rate_hz , 'wlx002e2d2091fa'=interface(LinuxWifiCollector = collector
(0.15=window_seconds(RssiFeatureExtractor = extractor
(3.0=presence_variance_threshold(PresenceClassifier = classifier

()start.collector
'!)motion test to around Walk( ...data RSSI of seconds 15 Collecting'(print
(15)sleep.time
()stop.collector

()get_samples.collector = samples
:samples if
)samples(extract.extractor = features
)features(classify.classifier = result
'dBm }f1.:mean.features{ :mean nRSSI}\)samples(len{ :Samples'f(print
,result(getattr{ :nVerdict}\f4.:variance.features{ :Variance'f(print
'})})result , "motion_level" →

```

خروجی این تست، توانمندی سیستم در درک واریانس محیطی را اثبات کرد:

```

30 :Samples
dBm 3.44- :mean RSSI
1678.13 :Variance
8927.14 :Energy Motion
ACTIVE.MotionLevel :Verdict

```

۳۰۲ پایش لحظه‌ای (Live Monitoring)

با اسکریپت live_monitoring_for_linux.py موفق شدیم پایش بی‌درنگ را پیاده‌سازی کنیم:

```

ABSENT.MotionLevel >= 000.0=var 0000.0=motion dBm0.52- =RSSI [14:51:27]
ACTIVE.MotionLevel >= 151.1455=var 2144.11071=motion dBm2.55- =RSSI [14:51:39]
PRESENT_STILL.MotionLevel >= 023.2=var 9508.3=motion dBm7.42- =RSSI [14:51:57]

```

۴۰۲ تحلیل استراتژیک: چرا از فازیک عبور کردیم؟

سیستم بالا عملکرد مطلوبی در تشخیص حضور داشت، اما دو خلأ اساسی ما را ملزم به ارتقای معماری کرد:

۱. محدودیت سیگنال: پارامتر RSSI اطلاعات دقیقی از تغییرات «فاز» موج را شامل نمی‌شود و برای اندازه‌گیری علائم حیاتی (نظیر تنفس) ناکارآمد است؛ ما به اطلاعات وضعیت کانال (CSI) نیاز داشتیم.
۲. گلوگاه نرم‌افزاری: زبان پایتون در پردازش بی‌درنگ ماتریس‌های سنگین CSI کند عمل می‌کند.

نتیجه‌گیری برای گذار: برای رفع این مشکلات، در فاز دوم به سراغ بردهای ESP32-S3 (جهت استخراج سخت‌افزاری CSI) و زبان قدرتمند Rust (برای پردازش بی‌درنگ) رفتیم.

۳ فاز دوم: شبکه مش CSI با زبان Rust (RuView V2)

در این مرحله، گره‌های ESP32 داده‌های خام CSI را از محیط استخراج کرده و به یک سرور مرکزی پر قدرت Rust ارسال می‌کنند.

۱.۳ راه‌اندازی محیط Rust و سرور پردازشی

ابتدا در پوشه v2 وابستگی‌های گیت دریافت و ابزارهای Rust کامپایل شدند:

```
recursive-- init-- update submodule git
server-sensing-densepose-wifi p- release-- build cargo
```

چالش خطای **OpenBLAS**: در اولین بیلد با خطا مواجه شدیم که دلیل آن عدم نصب کتابخانه محاسبات ماتریسی C بود. با نصب آن مشکل کامپایل حل شد:

```
config-pkg dev-libopenblas install get-apt sudo && update get-apt sudo
```

جهت ارزیابی پردازش، بنچمارک سیستم نشان داد که سرور Rust هزار فریم CSI را در کسری از ثانیه محاسبه می‌کند. همچنین برای تست بدون برد سخت‌افزاری از حالت شبیه‌ساز استفاده گردید:

```
benchmark-- server-sensing/release/target./
simulate source-- server-sensing/release/target./
```

۲.۳ آماده‌سازی ESP-IDF و پیکربندی گره‌های ESP32

برای توسعه سورس کد RuView، نسخه پایدار ESP-IDF v5.4 نصب گردید (نسخه v5.2 دارای باگ بود):

```
esp~/ cd && esp~/ p- mkdir
git.idf-esp/espressif/com.github://https recursive-- 4.v5 b- clone git
esp32s3 sh.install./ && idf-esp cd
```

۱.۲.۳ روش سنتی کامپایل (menuconfig)

ابتدا متغیرهای وای‌فای و سرور به روش دستی تنظیم و فلش شدند:

```
sh.export/idf-esp/esp/$HOME .
ttyACM0/dev/ 666 chmod sudo
esp32s3 target-set py.idf
```

```
menuconfig py.idf
monitor flash ttyACMO/dev/ p- py.idf && flash-erase ttyACMO/dev/ p- py.idf
```

۲۰۲۰۳ روش نوین و سریع (Provision.py)

کامپایل دستی برای یک شبکه مش زمان‌بر بود. لذا روش بهینه نگارش مستقیم در پارتیشن NVS به‌کار گرفته شد. با این اسکرپت، بدون نیاز به بیلد مجدد، در کمتر از ۱ ثانیه اطلاعات در فلش قرار می‌گرفت:

Provision.py

```
1 Node #
" password-- "WiFi-Sharif" ssid-- ttyACMO/dev/ port-- py.provision python
3 total-tdm-- 0 slot-tdm-- 1 id-node-- 87.2.27.172 ip-target-- ➔

2 Node #
" password-- "WiFi-Sharif" ssid-- ttyACM1/dev/ port-- py.provision python
3 total-tdm-- 1 slot-tdm-- 2 id-node-- 87.2.27.172 ip-target-- ➔
```

۳۰۳ چالش‌های استقرار شبکه و راهکارها

۱. مسدود بودن پکت‌های **UDP**: فایروال لینوکس دسترسی پورت ۵۰۰۵ را می‌بست.
راهکار: باز کردن پورت با `sudo ufw allow 5005/udp`

۲. اختلاف زمانی (**Timestamp Spread**) در مش چندگانه‌ای: تداخل پکت‌های ۳ گره باعث خطای ادغام می‌شد.
راهکار: استفاده از مکانیزم تسهیم زمانی (TDM) در لایه سرور:

```
package-- release-- run cargo =200000WDP_TDM_SLOT_US =3WDP_TDM_SLOTS
lan-insecure-udp-- 0.0.0.0 bind-udp-- -- server-sensing-densepose-wifi ➔
esp32 source-- ➔
```

۳. نفوذ امواج از دیوار (**Through-Wall**): امواج ۴۰۲ گیگاهرتز اشخاص حاضر در راهرو را نیز سنس می‌کردند.
راهکار: افزایش آستانه فیلتر (`WDP_MOTION_THRESHOLD=1.5`) برای حذف نویزهای بیرونی.

۴۰۳ تحلیل استراتژیک: چرا نیازمند فاز سوم شدیم؟

فاز دوم به ما دقت بی‌نظیر CSI و سرعت سرور Rust را داد، اما یک معماری ذاتاً **«توزیع‌شده و وابسته»** بود:

- گره‌های ESP32 فاقد درک مستقل بودند و تنها نقش آنتن را بازی می‌کردند.
- قطع شدن سرور لینوکسی به معنای از کار افتادن کل سیستم حسگری بود.
- همگام‌سازی زمانی در شبکه‌های پرنویز DHCP و وای‌فای خانگی بسیار دشوار است.

نتیجه‌گیری برای گذار: اگر بخواهیم این سیستم را در یک خانه هوشمند عادی مستقر کنیم، نیازمند سیستمی مستقل هستیم که خودش داده را پردازش کند و مستقیماً به داشبوردهای استاندارد (مثل Home Assistant) متصل شود. این نیاز، ما را به پیاده‌سازی مفهوم ISAC متمرکز در لبه (فاز سوم) هدایت کرد.

۴ فاز سوم: پردازش لبه متمرکز (Edge ISAC) با ESpectre

در فاز نهایی، با استفاده از پلتفرم ESpectre و ابزار ESPHome، سیستم استخراج و پردازش CSI را تماماً در داخل چیپ ESP32-S3 متمرکز کردیم. این گره، ترافیک استاندارد تولید کرده، بازتاب آن را می‌خواند، تحلیل می‌کند و خروجی نهایی را به صورت مستقیم تحویل می‌دهد.

۱۰۴ کدنویسی بدون C++ (Zero-Bloat Architecture)

در این معماری تمام تنظیمات توسط YAML نوشته شده و ESPHome کدهای بهینه (با مصرف تنها ۷۰ کیلوبایت فلش) تولید می‌کند.

```
venv venv m- python3
activate/bin/venv source
esphome install pip
```

۱۰۱۰۴ پی‌کردنی سخت‌افزاری برد

فایل‌های secrets.yaml و espectre-s3.yaml جهت فعال‌سازی Octal PSRAM تنظیم گردید:

espectre-s3.yaml

```
:esp32
ESP32S3 :variant
:framework
idf-esp :type
:sdkconfig_options
"y" :CONFIG_SPIRAM
"y" :CONFIG_SPIRAM_MODE_OCT
"y" :CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG
```

نکته کلیدی سخت‌افزار: برای ارتباط صحیح ttyACM0، کابل USB باید حتماً به درگاه Type-C سمت چپ برد متصل شود. فریم‌ورک با دستورات زیر روی برد فلش گردید:

```
erase_flash py.esptool
yaml.dev-s3-espectre/examples run esphome
```

۲۰۴ الگوریتم‌های پردازشی در لبه

۱. الگوریتم **MVS**: محاسبه واریانس فضایی پس از ۱۳ ثانیه کالیبراسیون برای ۱۲ ساب کریر با بالاترین SNR.
۲. الگوریتم یادگیری ماشین (**ML**): اجرای یک شبکه عصبی پرسپترون (MLP) با ساختار $9 \rightarrow 32 \rightarrow 16 \rightarrow 1$ در تراشه، که بوت زیر ۳ ثانیه داشته و نیاز به کالیبراسیون ندارد.

۳۰۴ یکپارچه‌سازی با Home Assistant در بستر Docker

برای نمایش گرافیکی داده‌ها، Home Assistant مستقر گردید. به دلیل تحریم‌ها، آینه‌های داکر داخلی ست شدند:

```
'EOF' << json.daemon/docker/etc/ tee sudo  
{ "ir.arvancloud.docker://https[" ":mirrors-registry" {  
    EOF  
docker restart systemctl sudo
```

کانتینر HA در حالت `net=host` اجرا شد تا با شبکه وای فای تداخلی نداشته باشد:

```
\ host=net-- privileged-- homeassistant name-- d- run docker  
  \ config:/homeassistant_config~/ v-  
  stable:assistant-home/assistant-home/io.ghcr
```

مدیریت چالش **IP** دینامیک: با استفاده از DHCP، پس از ری‌استارت روتر ممکن است IP برد تغییر کرده و در داشبورد Unavailable شود. راه‌حل این بود که لاگ‌های زنده برد را باز کرده (esphome logs)، آدرس جدید را خوانده و در تنظیمات Home Assistant گزینه **Reconfigure** را انتخاب کنیم تا بدون خرابی داشبورد، سنسورها متصل شوند.

۵ جمع‌بندی و نتیجه‌گیری نهایی مسیر تکامل

با گذر از این سه فاز عملیاتی، ما از یک اسکریپت ساده پایتون برای خوانش RSSI (با خطای بالا)، به یک شبکه مش توزیع‌شده قدرتمند با Rust رسیدیم که امکانات نظارت حرفه‌ای در فضاهای وسیع را فراهم می‌کند. در نهایت، با استقرار پلتفرم ESpectre، فناوری پیچیده ISAC را بومی‌سازی کرده و آن را در قالب یک گره کاملاً مستقل در لبه (Edge) و با قابلیت اتصال سریع به پنل خانه‌های هوشمند خلاصه کردیم.

انتخاب بین فاز دوم (RuView) (۷۲ و فاز سوم (ESpectre) در پروژه‌های آتی، بستگی مستقیم به مقیاس دارد: برای محیط‌های تجاری وسیع با نیاز به پوشش Non-Line-of-Sight، معماری فاز دوم (سرور مرکزی) ایده‌آل است؛ اما برای تجهیزات خانگی توکار و ارزان، معماری فاز سوم بلامنازع خواهد بود.